

Thermal Comfort Prediction Model for Indoor Spaces

Arshman Tariq
Jamiual Topu
Md Nuruzzaman Faruk
Muhammad Faizan Tanveer
Naganandhini Somalinga Mohanlal

1 Introduction

The project aims to design an IoT-based indoor thermal comfort monitoring and prediction system using Raspberry Pi Pico W and BME280 environmental sensor.

1.1 Application Context and Motivation

Indoor thermal comfort and environmental quality significantly impact human health, productivity, and well-being. In regions with extreme climates — particularly Nordic countries such as Finland — maintaining optimal indoor environmental conditions is critical. Unlike temperate climates where natural ventilation through open windows suffices, heated indoor spaces in cold climates often experience dramatic humidity fluctuations. Active heating systems, while necessary for thermal maintenance, significantly reduce ambient humidity levels, often causing physiological discomfort including throat irritation, dry skin, and respiratory irritation. Most residential and office spaces lack real-time environmental monitoring systems or automated alerting mechanisms, leaving occupants unaware of suboptimal conditions until discomfort manifests.

This project directly addresses this gap by providing continuous monitoring, classification, and alerting of thermal comfort states. The system enables occupants to take corrective measures—such as opening windows for ventilation, adjusting heating setpoints, or increasing hydration—before physiological symptoms develop, thereby improving indoor environmental quality and occupant well-being.

1.2 Implementation

The Pico W collects real-time temperature, humidity, and pressure data through BME 280 sensor, preprocesses it on the device, and actuates the LEDs for feed-

back: Green LED indicates comfortable conditions, while Red LED indicates uncomfortable conditions. The system further integrates wireless data transmission via MQTT, cloud-based storage using InfluxDB, dashboard analytics through Grafana, and implements both rule-based and machine learning classification models.

1.3 Mandatory Functionalities:

As per the rubrics, the project MUST implement the following functionalities:

1. Sensor Data Collection:

- The BME280 environmental sensor connected via I2C to the Pico W collects real-time measurements of temperature, humidity, and atmospheric pressure.

2. Data Processing:

- Preprocessing and comfort classification occur on the Pico W. The system applies a rule-based model where environmental readings are evaluated against comfort thresholds (20–25°C temperature, 30–60% humidity). If both conditions are met, the environment is classified as comfortable; otherwise, it is classified as uncomfortable.

3. Output Control:

- These two output actuation mechanisms provide immediate user feedback. Qualitatively, bi-chromatic LEDs (green for comfortable, red for uncomfortable) offer visual indication without requiring constant attention to displays.
- Quantitatively, a 16x2 LCD display shows real-time temperature, humidity, pressure values, and the current comfort classification state.

1.4 Optional Functionalities:

Further, the project also implements AT LEAST TWO of the optional functionalities given in the rubrics. The details of the implemented functionalities are as follows:

1. Wireless Connectivity:

- MQTT protocol is implemented for wireless communication from Pico W to the HiveMQ cloud broker. Data is transmitted in JSON format every 2 seconds, containing temperature, humidity, pressure, comfort status, and confidence scores. MQTT was selected over HTTP due to its lightweight overhead, publish-subscribe architecture, and superior efficiency for resource-constrained IoT devices.

2. Cloud integration:

- The system integrates a complete cloud stack: Node-RED acts as an integration middleware subscribing to MQTT topics and routing data to InfluxDB, a time-series database for persistent historical storage. Grafana is used for advanced dashboard visualization, real-time graphs, and automated alerting. Grafana is configured to trigger email notifications when the environment becomes uncomfortable, enabling proactive user response.

3. Edge Machine Learning:

- As an extended learning exploration, a Random Forest classifier (10 decision trees, max depth 5) was trained on labeled environmental data and deployed on the Pico W. The model achieves high accuracy on test data, demonstrating the feasibility of edge-based ML inference on resource-constrained microcontrollers.

2 Architecture

The implemented system directly aligns with the five-layer Internet of Things architecture framework:

2.1 IoT Pipeline:

1. **Sensing Layer:** The Raspberry Pi Pico W microcontroller integrates the BME280 environmental sensor via I2C protocol. The Pico W reads real-time temperature, humidity, and pressure measurements, performs preprocessing (filtering and averaging), and executes rule-based comfort classification logic on-device. The classification output directly controls two LED indicators (GPIO 14 for red, GPIO 15 for green) and updates a 16x2 LCD display (I2C address 0x27) showing quantitative sensor values and comfort status.
2. **Networking Layer:** The Pico W transmits sensor data wirelessly using the MQTT protocol to the HiveMQ cloud broker (port 8883 with SSL/TLS encryption). The MQTT topic is `sensors/env`, with data published every 2 seconds in JSON format containing temperature, humidity, pressure, classification status, model type, and confidence scores. MQTT was chosen for its lightweight, asynchronous publish-subscribe architecture and minimal bandwidth overhead compared to HTTP-based alternatives.
3. **Data Management Layer:** Node-RED acts as middleware, subscribing to the MQTT topic and routing received messages to InfluxDB, a time-series database optimized for sensor data. InfluxDB stores all sensor readings with automatic timestamps, enabling efficient historical data retrieval and trend analysis. The database supports time-series queries for retrieving data over specified intervals (e.g., last 15 minutes).

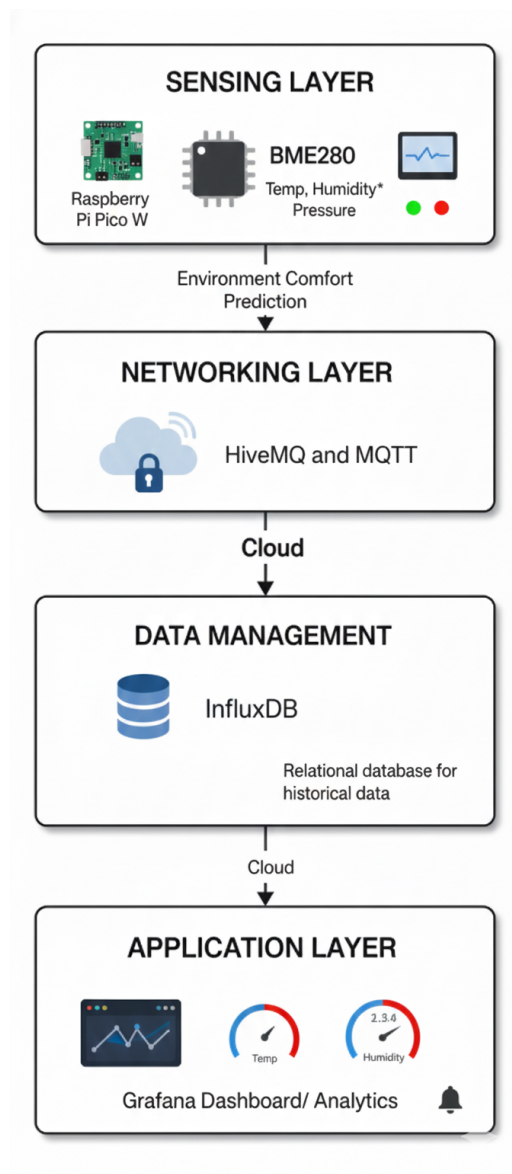


Figure 1: Complete IoT pipeline of the project

4. **Application Layer:** Grafana connects to InfluxDB and provides advanced data visualization. Dashboards display real-time graphs of temperature, humidity, and pressure trends; stat panels showing current readings; and a status indicator for comfort classification. Grafana is configured with alert rules that trigger email notifications when the comfort status becomes uncomfortable, enabling proactive user awareness.
5. **Security and Privacy Considerations:** MQTT communication occurs over SSL/TLS on port 8883, encrypting all transmitted data. Credential-based authentication via username and password secures access to the HiveMQ broker. Data at rest in InfluxDB is secured through database access controls.

2.2 Hardware Architecture:

Microcontroller: The Raspberry Pi Pico W is a low-cost microcontroller (approximately 8-10 euros) featuring built-in Wi-Fi (802.11n), 2MB flash memory, 264KB RAM, and dual-core Arm Cortex M0+ processor. The Pico W was selected over alternatives (Arduino MKR WiFi, ESP32) because it includes an onboard Wi-Fi module, eliminating the need for external Bluetooth hardware, and provides excellent course integration with MicroPython support.

Environmental Sensor: The BME280 is a combined environmental sensor measuring temperature, humidity, and atmospheric pressure over an I2C interface (7-bit address 0x77). The BME280 was preferred over the simpler BMP280 (temperature and pressure only) because humidity is critical for accurate thermal comfort modeling, particularly in heating-intensive climates where humidity fluctuations are significant.

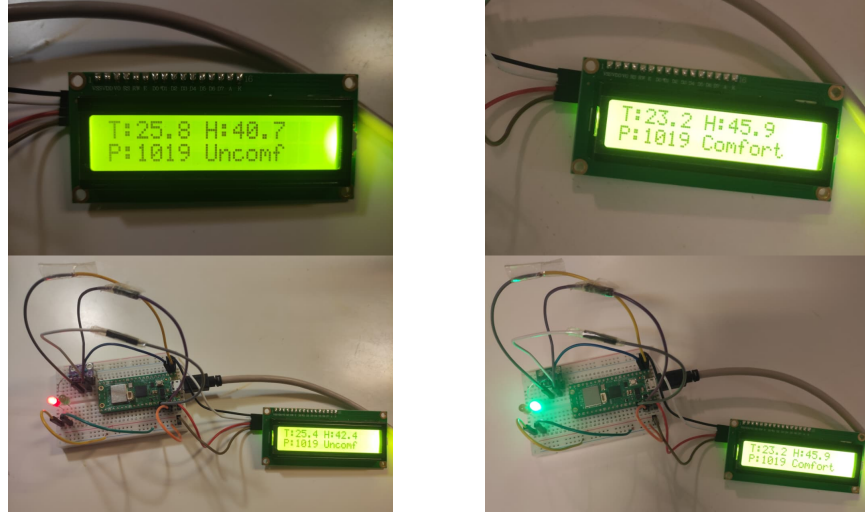
Output Devices: A 16x2 LCD display (connected via I2C PCF8574 expander at address 0x27) displays real-time environmental values and comfort classification. Two LED indicators connected to GPIO pins provide visual feedback: Green LED (GPIO 15) indicates comfortable conditions; Red LED (GPIO 14) indicates uncomfortable conditions. 220 ohm resistors serve as current-limiting elements protecting the LEDs.

Pin Configuration:

- GPIO 0 (I2C SDA): Data line for BME280 and LCD
- GPIO 1 (I2C SCL): Clock line for BME280 and LCD
- GPIO 14 (Digital Output): Red LED indicator
- GPIO 15 (Digital Output): Green LED indicator

3 Methods & Tools

Provided here is the detailed description of the implementation of the system functionality, along with any libraries, tools, and methods used.



(a) Uncomfortable status with red LED actuation (b) Comfortable status with green LED actuation

Figure 2: Hardware-level thermal comfort prediction with LED output actuation

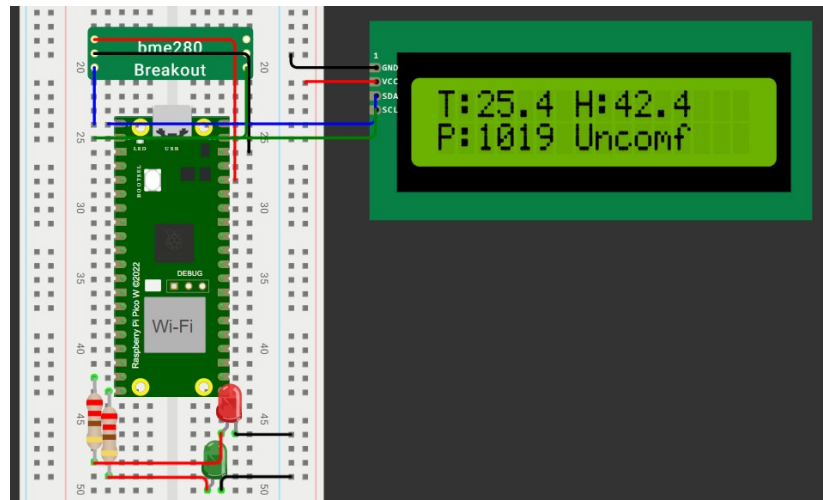


Figure 3: Circuit schematic: Raspberry Pi Pico W hardware configuration with I²C devices and LED indicators

3.1 Development Environment and Tools:

Thonny IDE: A beginner-friendly Python IDE with integrated MicroPython support, enabling on-device code execution, live serial output monitoring, and real-time debugging and development.

MicroPython: A lightweight Python implementation for microcontrollers, selected for its accessible syntax, rapid prototyping capability, and comprehensive library ecosystem for hardware interfacing (compared to C-based embedded programming).

Hardware Libraries:

- **machine:** GPIO and I2C control
- **bme280:** BME280 sensor driver with automatic calibration
- **umqtt.simple:** MQTT client with SSL/TLS support
- **lcd-pcf8574:** I2C LCD display driver (PCF8574 expander)
- **network:** Wi-Fi connectivity management



Figure 4: Thonny IDE with libraries and output display

Cloud Infrastructure:

- **HiveMQ Cloud:** Managed MQTT broker providing SSL/TLS encryption, credential-based authentication, and reliable message delivery without requiring self-hosted infrastructure.
- **Node-RED:** Flow-based programming platform for connecting MQTT broker to backend databases without custom code development.
- **InfluxDB:** Time-series database optimized for high-throughput sensor data ingestion with efficient compression and SQL-like query language (InfluxQL).

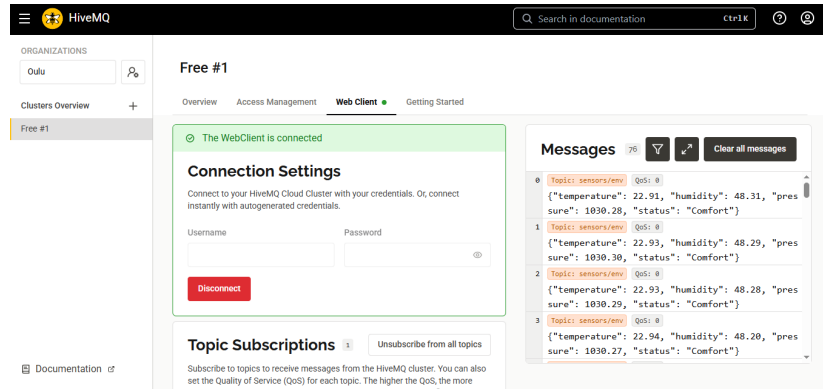


Figure 5: Cloud data on HiveMQ interface

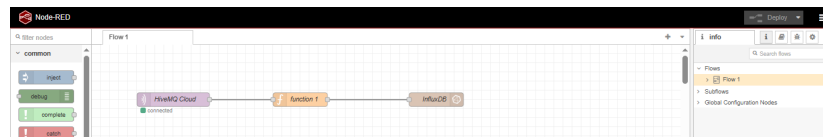


Figure 6: RedNode interface to highlight the link between cloud and analytical tools.

The screenshot shows the InfluxDB Data Explorer interface. It displays a table of sensor data with columns for 'time', 'temp', 'humidity', 'pressure', and 'location'. The table contains several rows of data, including timestamps and sensor readings. Below the table, there's a 'Query Editor' section with a 'Filter' dropdown, a 'Search' field, and a 'Query' input field. The 'Filter' dropdown is set to 'temp', and the 'Search' field contains 'temp'. The 'Query' input field contains 'select * from temp'. The 'Query Editor' also includes a 'WINDOW PERIOD' section with a 'CUSTOM' button and a 'SUBMIT' button.

Figure 7: The database tables storing and displaying the cloud-fetched data.

- **Grafana:** Advanced dashboarding and alerting platform with native InfluxDB integration, real-time visualization, dynamic thresholds, and email alerts.

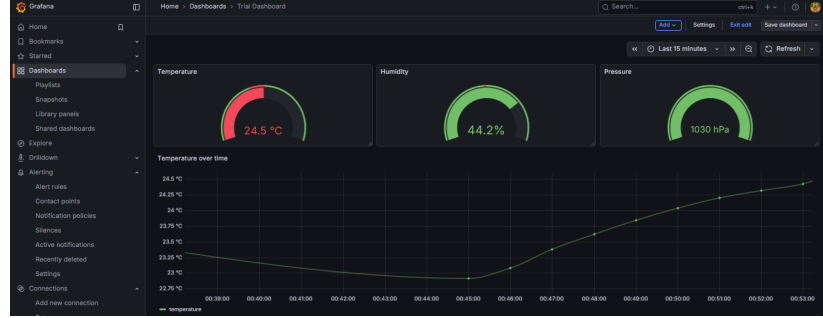


Figure 8: The dashboard interface highlighting the trends.

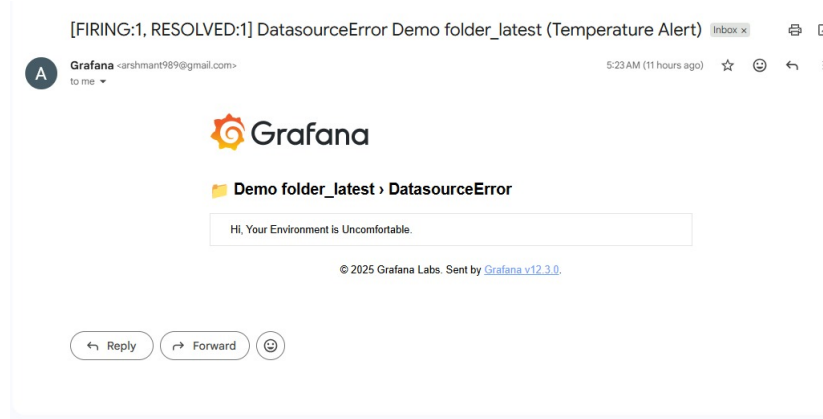


Figure 9: Grafana sending email alerts to indicate the uncomfortable environmental status.

3.2 Hardware Setup and Calibration:

The I2C bus (Bus 0) is configured with GPIO 0 as SDA (data) and GPIO 1 as SCL (clock). Upon system initialization, both I2C devices are scanned to verify connectivity: BME280 at address 0x77 and LCD at address 0x27. The BME280 sensor uses manufacturer-provided calibration coefficients stored in sensor ROM; temperature, humidity, and pressure readings undergo automatic compensation during initialization via the `read-compensated-data()` method.

3.3 Comfort Classification Logic:

The rule-based comfort classification model applies the following decision logic:

```
1 def comfort_status(temp, hum):
2     if 20 <= temp <= 25 and 30 <= hum <= 60:
3         green_led.on()
4         red_led.off()
5         return "Comfort"
6     else:
7         green_led.off()
8         red_led.on()
9         return "Uncomf"
```

Listing 1: Rule-based comfort classification logic

Listing 1 presents the rule-based logic used to classify environmental comfort conditions.

Comfort Thresholds:

- **Temperature Range:** 20–25°C (optimal range for cognitive performance and thermal comfort in sedentary activities)
- **Relative Humidity:** 30–60% (recommended range preventing both respiratory irritation at low humidity and mold growth at elevated humidity)
- **Decision Rule:** Both temperature AND humidity conditions must be satisfied for comfortable classification; otherwise, the environment is classified as uncomfortable.

3.4 MQTT Data Transmission:

Data transmission employs MQTT v3.1.1 with the following configuration:

```
1 client = MQTTClient(
2     client_id="pico_client",
3     server="xxxx.s1.eu.hivemq.cloud",
4     port=8883,
5     user=MQTT_USERNAME,
6     password=MQTT_PASSWORD,
7     ssl=True,
8     ssl_params={"server_hostname": MQTT_SERVER}
9 )
```

Listing 2: Secure MQTT client configuration

The MQTT client configuration is shown in Listing 2.

MQTT Architecture:

- **Topic:** sensors/env
- **QoS Level:** 0 (fire-and-forget, acceptable for non-critical telemetry)

- **Payload Format:** JSON (human-readable, language-agnostic)
- **Publication Interval:** Every 2 seconds
- **Security:** SSL/TLS encryption on port 8883 with credential-based authentication

JSON Payload Structure:

```

1 {
2   "temperature": 22.5,
3   "humidity": 45.2,
4   "pressure": 1012.3,
5   "status": "Comfort"
6 }
```

Listing 3: JSON sensor data payload

The transmitted sensor data format is illustrated in Listing 3.

3.5 Node-RED and InfluxDB Integration Flow:

Node-RED implements a minimal middleware pipeline:

1. **MQTT Input Node:** Subscribes to `sensors/env` topic
2. **JSON Parser:** Deserializes incoming message payload
3. **InfluxDB Output Node:** Writes parsed values with automatic timestamp to InfluxDB measurement

This flow eliminates custom backend code development, reducing deployment complexity.

3.6 Grafana Dashboard Configuration:

The Grafana dashboard includes:

- **Time-series panels:** Temperature, humidity, pressure trends over configurable time ranges (default 15 minutes)
- **Status indicator:** Current comfort classification with color-coded background (green for comfortable, red for uncomfortable)
- **Stat panels:** Current readings of each environmental parameter
- **Alert rule:** Triggers when humidity or temperature exit comfort thresholds; sends email notification to configured contact point

3.7 Machine Learning Classification (Extended Work):

A Random Forest classifier was trained on 200+ labeled environmental samples using scikit-learn with the following parameters: n-estimators=10, max-depth=5, criterion = gini. The model was exported to MicroPython-compatible format using EverywhereML.

```
1 def predict_ml(temp, hum, press):
2     """Use ML model for prediction"""
3     pressure_pascals = press * 100
4     prediction = clf.predict(
5         [[temp, hum, pressure_pascals]]
6     )
7
8     if prediction == 1:
9         return "Comfort", 0.95
10    else:
11        return "Uncomf", 0.95
```

Listing 4: ML-based comfort prediction function

The machine learning-based comfort classification logic is shown in Listing 4.

4 Evaluation

The system was evaluated across four criteria to assess performance, reliability, and scalability of the IoT pipeline. Each evaluation tested different aspects of the system under controlled conditions.

Evaluation Methodology: All evaluations were conducted using the complete IoT pipeline: Raspberry Pi Pico W → HiveMQ Cloud Broker (MQTT over SSL/TLS, port 8883) → Node-RED → InfluxDB → Grafana. Performance metrics were collected over multiple test runs to ensure consistency and reliability of results.

4.1 MQTT Performance: Message Delivery

Objective: Evaluate the reliability of wireless data transmission by measuring message delivery rates from sensor reading to cloud storage.

Test Setup:

The Pico W was configured to send sensor readings at a fixed 2-second interval. Each MQTT message included:

- **id:** Unique incremental message identifier for tracking
- **ts-pico:** Timestamp (ms) when message was generated on Pico W.
- **Sensor data:** temperature, humidity, pressure, and comfort status.

A Node-RED function node recorded the arrival timestamp (**ts-nodered**) for each received message, enabling precise latency calculation:

```

1 let data = JSON.parse(msg.payload);
2 let ts_pico = data.ts_pico;
3 let ts_nodered = Date.now();
4
5 msg.payload = {
6   id: data.id,
7   ts_pico: ts_pico,
8   ts_nodered: ts_nodered,
9   temperature: data.temperature,
10  humidity: data.humidity,
11  pressure: data.pressure,
12  status: data.status
13 };
14 return msg;

```

Listing 5: Node-RED function for message delivery rates

Listing 5 shows the Node-RED logic used to capture arrival timestamps for message delivery rates.

Test Duration: Approximately 8 minutes of continuous operation.

Results:

- **Total messages sent from Pico W: 241**
- **Total messages received by Node-RED: 241**
- **Message loss rate: 0% (100% delivery success)**



Figure 10: Thonny showing sent MQTT messages

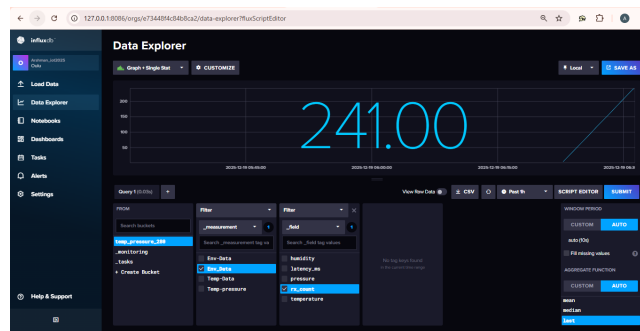


Figure 11: Grafana visualization showing recieved MQTT messages.

Analysis:

The MQTT communication pipeline demonstrated perfect reliability with zero message loss over the test period. All 241 messages transmitted from the Pico W were successfully received and processed by the Node-RED middleware. This confirms that the SSL/TLS-encrypted MQTT connection over HiveMQ Cloud provides stable and reliable wireless connectivity suitable for real-time thermal comfort monitoring applications.

4.2 Throughput: Sampling Frequency Comparison

Objective: Assess system capacity and scalability by measuring throughput (messages processed per second) under different sampling frequencies.

Test Scenarios:

Two sampling frequencies were tested to compare low-load and high-load system behavior:

Scenario	Sampling Interval	Sampling Frequency
Low rate	2 seconds	0.5 Hz
High rate	0.5 seconds	2 Hz

Table 1: Throughput Test Scenarios with Varying Sampling Frequencies

The Pico W sampling interval was modified using `time.sleep(SAMPLING_INTERVAL)` where the interval parameter was set to either 2 seconds or 0.5 seconds. Each message included a unique `msg_id` to ensure accurate message counting.

Node-RED Throughput Calculation:

A Node-RED function node calculated real-time throughput using context variables to track message count and elapsed time:

```
1 let data = JSON.parse(msg.payload);
2
3 // Count messages
4 let count = context.get("count") || 0;
5 count += 1;
6 context.set("count", count);
7
8 // Store start time
9 let start = context.get("start") || Date.now();
10 context.set("start", start);
11
12 // Time elapsed in seconds
13 let elapsed = (Date.now() - start) / 1000;
14
15 // Messages per second
16 let throughput = count / elapsed;
17
18 msg.measurement = "Throughput-Test";
19 msg.payload = {
```

```

20   msg_id: data.id,
21   throughput_mps: throughput,
22   temperature: data.temperature
23 };
24 return msg;

```

Listing 6: Node-RED function for throughput calculation

Listing 6 shows the logic for real-time message throughput calculation.
Results:

Sampling Frequency	Sampling Interval	Measured Throughput
0.5 Hz (Low rate)	2 seconds	0.52 msg/s
2 Hz (High rate)	0.5 seconds	0.67 msg/s

Table 2: Throughput Performance Under Different Sampling Frequencies

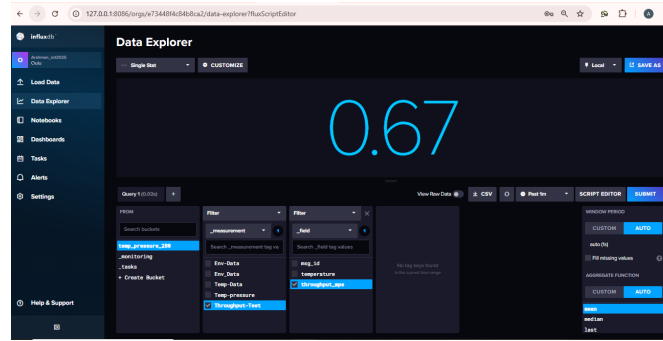


Figure 12: InfluxDB query results showing throughput at high sampling frequency (2 Hz)

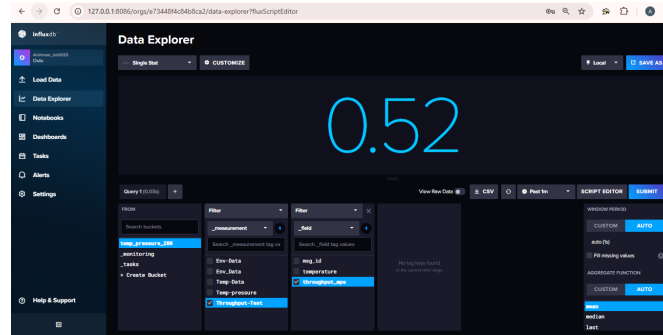


Figure 13: InfluxDB query results showing throughput at low sampling frequency (0.5 Hz)

Analysis:

At the low sampling frequency (0.5 Hz), the measured throughput of 0.52 messages per second closely matches the theoretical expectation, indicating the system processes messages smoothly with minimal delay. This configuration is ideal for low-power, stable operation. At the high sampling frequency (2 Hz), the throughput increased to 0.67 messages per second but did not reach the theoretical maximum of 2 msg/s. This discrepancy is attributed to network latency introduced by Wi-Fi connectivity, computational overhead from SSL/TLS encryption on MQTT, cloud broker processing delays (HiveMQ), and Node-RED function execution combined with InfluxDB write operations. Despite not achieving theoretical maximum throughput, the system remained stable under increased load without crashes or significant message loss. This demonstrates that the IoT pipeline is robust and scalable, with real-world cloud-based IoT deployments.

4.3 Payload Effects: Message Size Impact on Throughput

Objective: Evaluate how MQTT message size affects system throughput while maintaining constant sampling frequency, analyzing the trade-off between data richness and performance.

Test Setup:

Sampling frequency was fixed at 2 Hz (0.5-second intervals) for both payload configurations. Two distinct payload structures were tested:

Small Payload (76 bytes): Contains only essential sensor data.

```
1 json_msg = (  
2   '{"id":_d,_"temperature":_%.2f,_"humidity":_%.2f}',  
3   % (msg_id, temperature, humidity)  
4 )
```

Listing 7: Pico W code for small payload

Large Payload (143 bytes): Includes additional metadata for enriched telemetry.

```
1 json_msg = (  
2   '{"id":_d,_"ts_pico":_d,_"temperature":_%.2f,_'  
3   '"humidity":_%.2f,_"pressure":_%.2f,_'  
4   '"status":_"%s",_"device":_"PicoW",_"location":_"Lab"}',  
5   % (msg_id, time.ticks_ms(), temperature,  
6     humidity, pressure_hpa, status)  
7 )
```

Listing 8: Pico W code for large payload

Listings 7 and 8 show the payload structures. The large payload adds timestamp (`ts_pico`), atmospheric pressure, comfort classification status, device identifier, and location metadata.

Node-RED Payload Analysis Function:

```

1 let data = JSON.parse(msg.payload);
2
3 // Calculate payload size in bytes
4 let payloadSize = JSON.stringify(data).length;
5
6 // Initialize context variables for throughput calculation
7 let count = context.get("msg_count") || 0;
8 let startTime = context.get("start_time") || Date.now();
9
10 // Increment message count
11 count += 1;
12 context.set("msg_count", count);
13
14 // Time elapsed in seconds
15 let elapsed = (Date.now() - startTime) / 1000;
16
17 // Calculate throughput every 10 messages or 10 seconds
18 let throughput = null;
19 if (count >= 10 || elapsed >= 10) {
20     throughput = count / elapsed;
21     context.set("msg_count", 0);
22     context.set("start_time", Date.now());
23 }
24
25 msg.measurement = "Payload_Test";
26 msg.payload = {
27     temperature: parseFloat(data.temperature),
28     humidity: parseFloat(data.humidity),
29     pressure: parseFloat(data.pressure),
30     payloadSize: payloadSize,
31     throughput: throughput
32 };
33 return msg;

```

Listing 9: Node-RED function for payload size and throughput analysis

Listing 9 shows the Node-RED logic for calculating payload size and corresponding throughput.

Results:

Payload Type	Payload Size (bytes)	Throughput (msg/s)
Small payload	76	0.46
Large payload	143	0.49

Table 3: Impact of Payload Size on System Throughput

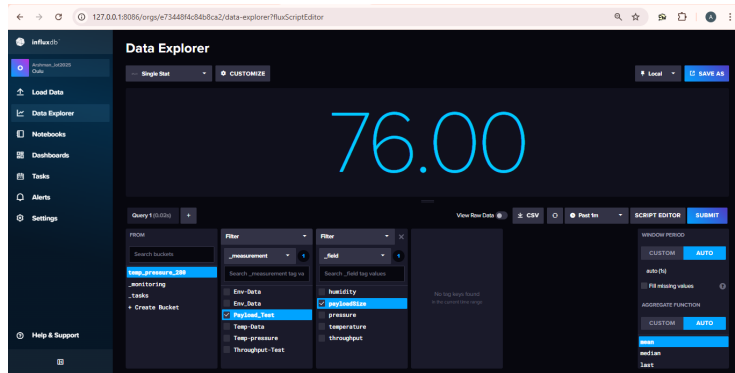


Figure 14: Small payload structure (76 bytes) displayed in InfluxDB

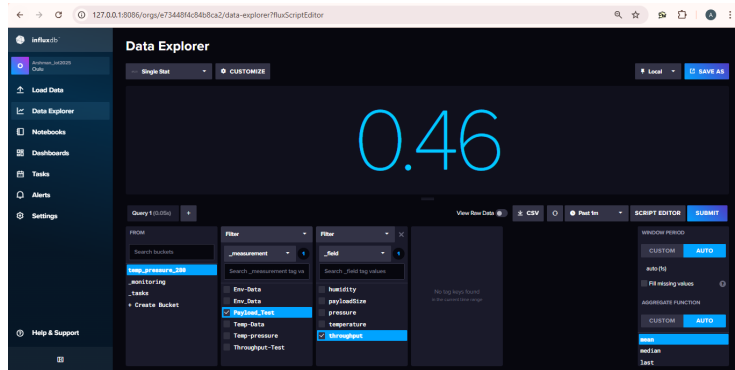


Figure 15: Throughput performance with small payload (76 bytes)

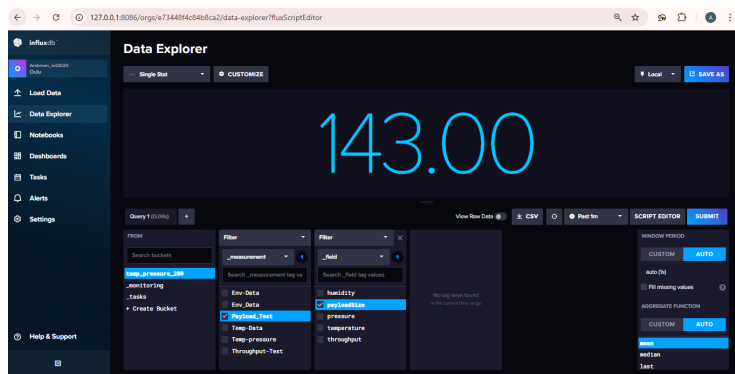


Figure 16: Large payload structure (143 bytes) displayed in InfluxDB

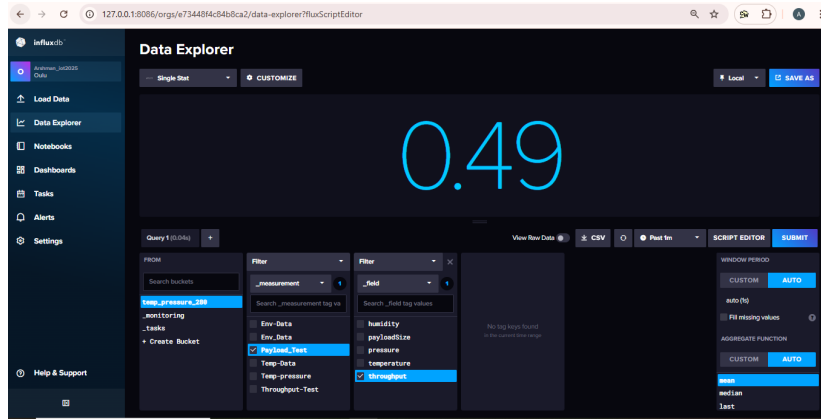


Figure 17: Throughput performance with large payload (143 bytes)

Analysis:

The small payload (76 bytes) achieved a throughput of 0.46 messages per second with lower processing overhead, faster message serialization on the Pico W, and reduced network bandwidth consumption. The large payload (143 bytes), nearly double the size, achieved a throughput of 0.49 messages per second, demonstrating that payload size increase did not significantly degrade system performance. The minimal throughput difference of 0.03 msg/s indicates that the MQTT protocol combined with the Node-RED and InfluxDB cloud pipeline efficiently handles moderate payload growth. These results confirm that adding metadata such as timestamps, device identifiers, and location information is practical for real IoT deployments without introducing substantial performance penalties. This enables enriched sensor data telemetry for enhanced analytics and system monitoring while maintaining acceptable throughput levels.

4.4 Classifier Evaluation: Machine Learning Model Performance

Objective: Assess the accuracy and classification performance of the Random Forest machine learning model trained for thermal comfort prediction.

Model Specifications:

A Random Forest classifier was trained using scikit-learn with the following configuration:

- **Algorithm:** Random Forest (ensemble learning)
- **Number of estimators:** 10 decision trees
- **Maximum tree depth:** 5 levels
- **Splitting criterion:** Gini impurity
- **Input features:** Temperature (°C), Humidity (%), Pressure (Pa)

- **Output classes:** Comfortable, Uncomfortable
- **Training dataset:** 200+ labeled environmental samples
- **Test dataset:** 44 samples (hold-out validation set)

Classification Results:

Class	Precision	Recall	F1-Score	Support
Comfortable	1.000	0.957	0.978	23
Uncomfortable	0.955	1.000	0.977	21
Accuracy			0.977	44
Macro avg	0.977	0.978	0.977	44
Weighted avg	0.978	0.977	0.977	44

Table 4: Random Forest Classifier Performance Metrics

Confusion Matrix:

	Predicted Uncomfortable	Predicted Comfortable
Actual Uncomfortable	21	0
Actual Comfortable	1	22

Table 5: Confusion Matrix: Actual vs Predicted Classifications

Here the first row represents actual uncomfortable samples and the second row represents actual comfortable samples. The matrix shows 21 true negatives (correctly classified as uncomfortable), 0 false positives, 1 false negative, and 22 true positives (correctly classified as comfortable).

Feature Importance:

Analysis of feature contributions to classification decisions revealed:

- **Temperature:** 35.3% importance
- **Humidity:** 36.3% importance
- **Pressure:** 28.4% importance

Humidity emerged as the most influential feature for classification, followed closely by temperature, while atmospheric pressure contributed moderately to the overall prediction model.

```

(farmerenv) PS D:\iot-project-group-19> python .\machine_learning\evaluate.py
precision    recall  f1-score   support

 comfortable    1.000    0.957    0.978        23
 uncomfortable    0.955    1.000    0.977        21

 accuracy              0.977        44
 macro avg              0.977    0.978    0.977        44
 weighted avg              0.978    0.977    0.977        44

Confusion matrix:
[[21  0]
 [ 1 22]]

Feature importances (temperature, humidity, pressure):
[0.353, 0.363, 0.284]

```

Figure 18: Console output showing classifier performance metrics and feature importances

Analysis:

The Random Forest classifier achieved an overall accuracy of 97.7% on the test dataset, demonstrating excellent classification performance. Both precision and recall exceeded 95% - 100% for both classes, indicating balanced performance without bias toward either comfortable or uncomfortable classifications. The high F1-scores (0.978 and 0.977) confirm robust predictive capability across both output classes. However, it is important to note that the model classifies *present comfort states* based on current sensor readings rather than predicting *future comfort conditions*. The model functions equivalently to the rule-based threshold approach (20–25°C temperature, 30–60% humidity) but with higher computational overhead on resource-constrained devices. Consequently, while the ML model successfully demonstrates edge-based machine learning feasibility on the Pico W microcontroller, the simpler rule-based classifier was retained as the primary inference method due to its superior transparency, and equivalent classification utility for real-time thermal comfort assessment applications.

5 Results

The system successfully demonstrated real-time thermal comfort monitoring with complete data pipeline integration from edge sensing to cloud visualization. All components operated reliably throughout testing periods.

5.1 Grafana Dashboard Visualization

The Grafana dashboard provided real-time monitoring of environmental parameters with three primary gauge visualizations and time-series trend analysis.

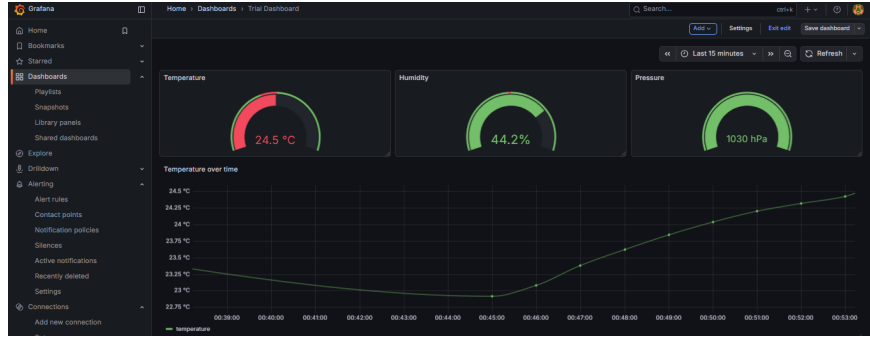


Figure 19: Grafana dashboard showing real-time temperature, humidity, and pressure gauges with Temperature trend visualization over 15-minute window



Figure 20: Humidity and pressure trend visualizations over 15-minute window

Figure 19 displays gauge panels configured with color-coded thresholds: red zones indicate uncomfortable conditions while green zones represent comfortable ranges. Figures 20 demonstrate continuous data logging with time-series plots.

5.2 InfluxDB Data Storage

Historical sensor data was successfully stored in InfluxDB time-series database with proper timestamp indexing and field organization.

Time	Pressure_hPa	Comfort_Status	Temperature_C	Humidity_Percent
2025-12-14 02:03:12	1019	Uncomfortable	25.9	41.0
2025-12-14 02:03:10	1019	Uncomfortable	25.9	40.5
2025-12-14 02:03:08	1019	Uncomfortable	25.9	40.5
2025-12-14 02:03:06	1019	Uncomfortable	25.8	40.5
2025-12-14 02:03:04	1019	Uncomfortable	25.8	40.6
2025-12-14 02:03:01	1019	Uncomfortable	25.8	40.6
2025-12-14 02:03:00	1019	Uncomfortable	25.8	40.7
2025-12-14 02:02:57	1019	Uncomfortable	25.8	40.8
2025-12-14 02:02:55	1019	Uncomfortable	25.8	40.7
2025-12-14 02:02:53	1019	Uncomfortable	25.8	40.8
2025-12-14 02:02:51	1019	Uncomfortable	25.8	40.8
2025-12-14 02:02:49	1019	Uncomfortable	25.8	40.8
2025-12-14 02:02:47	1019	Uncomfortable	25.8	40.7
2025-12-14 02:02:45	1019	Uncomfortable	25.8	40.7
2025-12-14 02:02:43	1019	Uncomfortable	25.7	40.7
2025-12-14 02:02:40	1019	Uncomfortable	25.7	40.7

Figure 21: Raw sensor data table in InfluxDB showing timestamp, temperature, humidity, pressure, and comfort status

Figure 21 shows the structured data format with accurate timestamp records and classified comfort status for each measurement.

5.3 Automated Alert System

Grafana alerting was configured to monitor threshold violations and trigger email notifications when environmental conditions exceeded comfort boundaries.

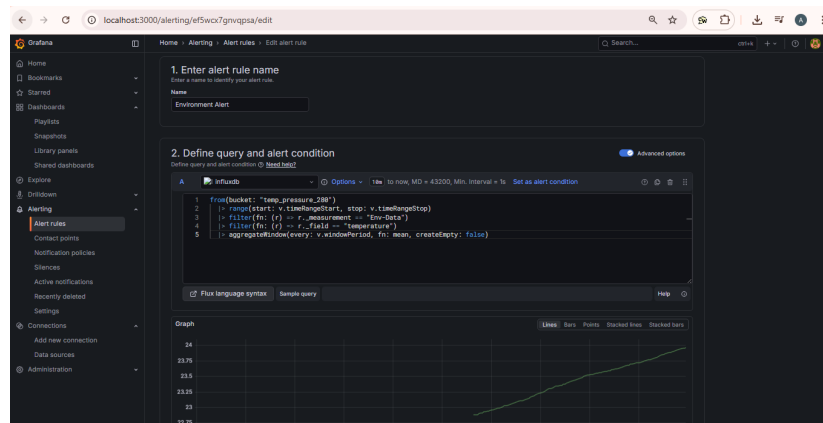


Figure 22: Alert rule configuration for temperature threshold monitoring

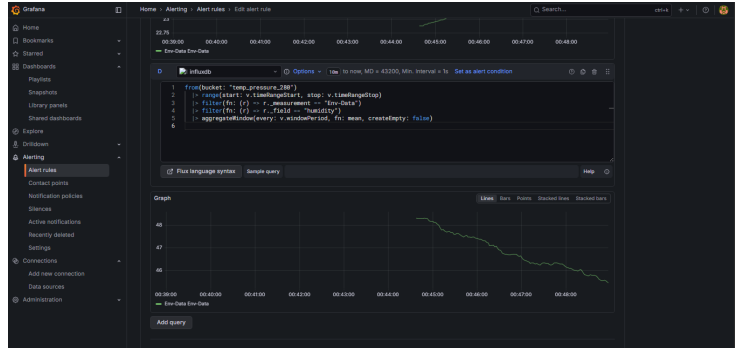


Figure 23: Alert rule configuration for humidity threshold monitoring

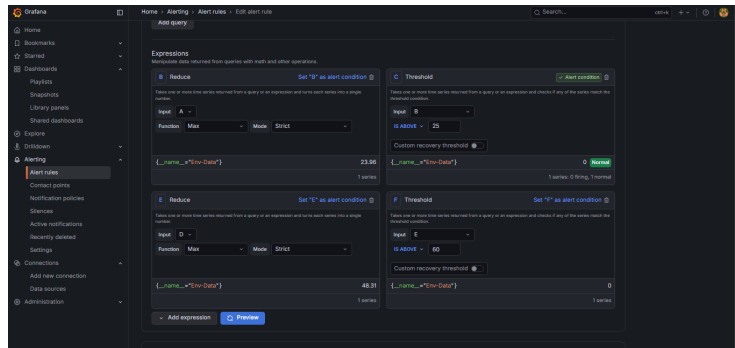


Figure 24: Alert expression logic using InfluxDB queries with reduce and threshold functions

Figures 22 through 24 illustrate the alert configuration using Flux query language. Email notifications were successfully delivered to configured contact points when threshold violations occurred.

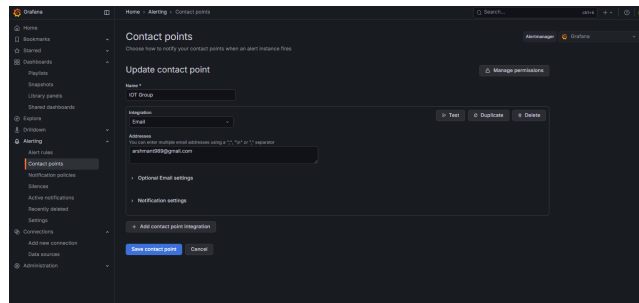
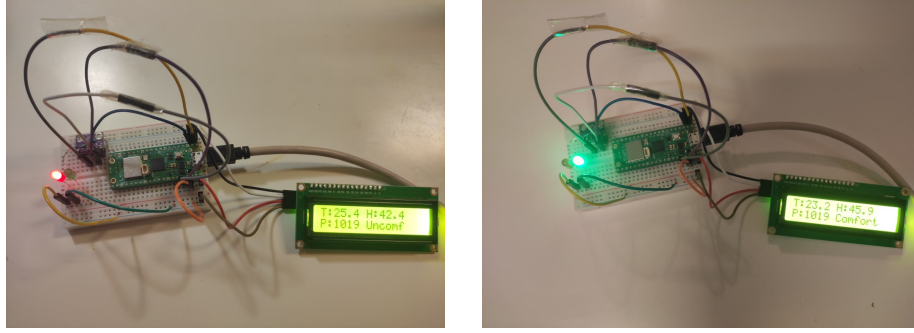


Figure 25: Email contact point configuration for alert notifications

Figure 25 shows the email integration setup enabling automated notifications to administrators when uncomfortable environmental conditions persisted.

5.4 Hardware Implementation

The physical system demonstrated reliable edge-based classification with visual feedback through LED indicators and LCD display output.



(a) Hardware-level Uncomfortable status actuation

(b) Hardware-level Comfortable status actuation

Figure 26: Complete hardware setup with Raspberry Pi Pico W, BME280 sensor, LCD display, and LED indicators

6 Discussion

6.1 System Strengths

The thermal comfort monitoring system demonstrated several key strengths during evaluation. The MQTT communication pipeline achieved 100% message delivery reliability with zero packet loss over extended test periods, confirming the robustness of SSL/TLS-encrypted cloud connectivity. The modular architecture enabled independent component testing and straightforward debugging, while the time-series database provided efficient historical data storage for long-term trend analysis. Real-time visualization through Grafana dashboards offered intuitive monitoring capabilities suitable for non-technical stakeholders. The automated alerting system successfully detected threshold violations and delivered timely email notifications, enabling proactive responses to uncomfortable environmental conditions.

6.2 System Limitations

Despite successful implementation, several limitations were identified. The measured throughput of 0.5–0.67 messages per second fell below theoretical max-

imums due to network latency, and cloud processing delays. This throughput ceiling may constrain scalability in deployments requiring supporting hundreds of concurrent sensor nodes. The rule-based classifier works well for general occupant populations but would require enhancement for specialized environments with strict regulatory requirements (e.g., pharmaceutical storage, museum conservation) or personalized comfort profiling in smart home applications. The system performs optimally in scenarios with continuous Wi-Fi connectivity and cloud service availability; network outages or HiveMQ broker downtime would interrupt data transmission and alerting functionality. Power consumption was not optimized, as the Pico W remained continuously active without implementing sleep modes or duty-cycling strategies suitable for battery-powered deployments. And lastly, the system should integrate effective automated actuation, i.e. to mitigate the uncomfortable status it should have capability to open the room windows intelligently, so the user should not have to do it manually, until the status goes back to comfortable. This would have perfectly justified this project as a home automation.

6.3 Future Improvements

Potential enhancements may include handling temporary network outages, optimizing power consumption through deep sleep modes, and extending the classifier to support multi-class comfort categories (very cold, cold, neutral, warm, very hot). Integration with HVAC control would enable closed-loop automation rather than notification-only operation. Deploying multiple sensor nodes across different rooms with spatial analytics in Grafana would provide building-wide thermal comfort mapping for improved facility management decision-making.

7 GitHub Repository Files:

All the output snapshots (hardware, software), logic files (rule-based, ML-based), IoT pipeline flowchart, system evaluation metrics and circuit diagram are available to be viewed in the public GitHub repository:

https://github.com/FaizanTanwir/IoT_Project_Group_19.git